경력기술서

HEO MIN | gjals0318@gmail.com | github.com/mingming-01

> 교육 과정과 팀 프로젝트에서 수행한 업무를 프로젝트별로 정리했습니다. 팀 프로젝트는 직접 담당한 기능과 그렇지 않은 범위를 구분해 적었습니다.

한눈에 보기

프로젝트	구분	맡은 일
Maple Growth Tracker	개인 (2026.09 ~ 진행 중)	기획부터 배포까지 전부 담당. 외부 API 데이터의 수집과 저장, 분석, 화면과 서버 구조 설계
SENTRA	ALEPH 캡스톤 3인 팀 (2026)	서버 백엔드: 로그인, 직원 가입, 기간제 권한 처리
Packtory_DE	ITWill 심화 팀 (2025.03 ~ 05)	직원/부서, 원자재/창고/입출고, 품질/근무 기능
Mr.Bean	ITWill 팀	원자재, 완제품, BOM, 입고와 LOT
Stock-Mate	ITWill 팀	카테고리와 재고 기능, 팀원 기능과 맞물리는 데이터 확인

주요 프로젝트

01 Maple Growth Tracker | 공개 API 데이터 수집과 분석 웹서비스

구분/기간	개인 프로젝트(기획, 개발, 배포), 2026.09 ~ 진행 중
사용 기술	Python, Flask, React 19, Vite, Chart.js, NEXON Open API, PostgreSQL(Supabase), SQLite, Gemini API, OpenRouter, Render
개요	공개 API의 캐릭터 데이터를 가져와 날짜별로 저장하고, 기간별 성장 분석과 캐릭터 비교, 상세 조회, AI 리포트로 보여주는 웹 서비스입니다. 현재 시점의 정보만으로는 알 수 없는 변화를 기록으로 쌓아 보여주는 것이 목적입니다. 배포해 지인 10명 내외가 사용했고, 받은 피드백과 건의사항 중 서비스 방향에 맞는 것을 일부 반영해 개선하고 있습니다. 기존 Flask 템플릿 화면은 남겨 두고 React 화면으로 개편했습니다.

설계 포인트

- 성장 지표(경험치 증가량, 일평균, 랭킹 변화)는 별도 계산 코드에서 산출하고, AI는 계산된 값과 기록을 문장으로 설명하도록 역할을 나눴습니다.
- 상세 정보(스탯, 장비, 심볼, 6차/HEXA, 링크, 유니온)는 탭을 선택했을 때만 따로 조회하고 서버에서 짧게 캐시합니다. 일부 조회가 실패해도 기본 정보와 성장 기록은 동작합니다.

담당 업무

- NEXON Open API 기반 기본 정보, 종합/월드/직업별 랭킹, 현재 전투력 조회와 화면용 데이터 가공
- 최근 7일 기록 기본 수집, 30일·90일 기록의 빈 날짜 10일 단위 보충, 저장 기록 재사용을 통한 중복 요청 감소
- 경험치 증가량, 일평균, 레벨 진행률, 랭킹 변화 계산과 타임라인·추이 그래프 표시, 다음 레벨 참고 일정 제공
- 두 캐릭터의 동일 기간 성장 비교, AI 성장 리포트 구현 (Gemini 우선, 조건에 따라 OpenRouter 대체)
- 결과 이미지/PDF 저장, 관심 캐릭터 관리, 관리자 페이지의 이용 로그 확인

문제 해결

- **없는 날과 실패한 요청의 구분** API가 아직 공개하지 않은 날짜와 일시적 오류로 조회하지 못한 날짜를 같게 취급하면 실제로 없던 성장 기록이 생깁니다. 두 경우를 나눠 저장하고, 이미 저장된 정상 기록은 빈 응답으로 덮어쓰지 않으며, 근거가 없는 구간은 값을 채우지 않았습니다.
- **근거 없는 예측 방지** 다음 레벨 예상은 최근 7일에 계산 가능한 변화가 충분하고 평균 성장량이 양수일 때만 구체적인 날짜를 보여줍니다. 그렇지 않으면 예측하지 않습니다.
- **배포 환경의 데이터 저장** 내 컴퓨터에서 되던 저장 방식이 배포 환경에서는 그대로 유지되지 않았습니다. DB 연결 값이 설정되면 PostgreSQL, 없으면 로컬 SQLite를 쓰도록 나누고, 실제로 저장되는지 직접 확인했습니다.

결과와 느낀 점

정상 데이터, 데이터가 없는 날짜, API 오류 상황을 나눠 테스트하고, 기본 자동 검사(GitHub Actions: Python 테스트, React 빌드, JavaScript 검사)를 연결했습니다. 브라우저 테스트(Playwright)는 별도 명령으로 실행합니다. 만든 서비스를 배포해 다른 사람이 쓰는 데까지 가 보니, 데이터가 맞는 지와 배포 환경이 개발 환경과 다른 점을 챙기는 일이 기능을 만드는 일만큼 중요했습니다.

링크 github.com/mingming-01/maple-growth-tracker | maple-growth-tracker.onrender.com

02 SENTRA | 사내망 다계층 접근통제 시스템

구분/기간	ALEPH 1기 캡스톤, 3인 팀 (서버 백엔드 담당), 2026.10 ~ 2026.11
사용 기술	Python, Flask, PyJWT, SQLite (화면은 React 기반으로 서버와 연결)
개요	직원의 역할과 소속, 접근하려는 자원에 따라 서버가 허용하거나 거부하는 사내 접근통제 시스템입니다. 직원 명부 기반 가입, 계정 상태 관리, 접근 정책, 감사 로그, 기간제 권한의 요청, 승인, 회수를 다룹니다. 팀 구성은 조장이 Azure 서버, 아키텍처, 프론트엔드, 챗봇을, 한 팀원이 네트워크 가상망 구축과 Packet Tracer 시연, allowList 확정을, 제가 서버 백엔드를 맡았습니다.

담당 업무

- 비밀번호 암호화 저장 기반의 로그인 인증과 토큰 발급, 비활성화된 계정의 접근 차단
- 관리자 등록 직원 명부 기준의 사원번호·일회용 초대 코드 확인 가입 절차와 코드 재사용 방지
- 직원, 사용자 데이터 저장 구조 설계, 마이페이지 처리
- 기간제 권한의 요청, 운영자 승인, 반려, 승인 권한의 만료·회수 흐름 구현
- 화면과 서버 간 요청, 응답 연결
- 담당 기능의 요구사항 정의서, DB 스키마, 플로우차트 작성 (팀 구글 드라이브 공유 문서)

문제 해결

- **승인 전후와 회수 후의 접근 결과** 요청을 제출했다고 바로 접근이 열리면 안 되므로, 승인 전에는 기존 정책을 유지하고 승인된 기간에만 예외 권한을 적용했습니다. 기간이 끝나거나 회수되면 원래대로 돌아가는지, 승인 전후와 회수 후에 접근 결과가 실제로 달라지는지 확인했습니다.
- **서버가 판단하는 접근통제** 화면에서 메뉴를 숨겨도 서버에 직접 요청하면 접근할 수 있으므로, 인증과 정책을 서버에서 확인하도록 했습니다. 접근이 거부되면 사유와 권한 요청 방법을 화면에 안내하도록 연결했습니다.

결과와 느낀 점

직원 가입, 초대 코드 재사용 방지, 비활성 계정 차단, 권한 요청, 승인, 회수 같은 주요 시나리오를 직접 실행해 확인했습니다. 권한은 만료와 회수까지 구현해야 관리가 끝난다는 점을 알게 됐습니다. 네트워크 장비 연동과 로그 수집, 알림 수신은 별도 환경에서 확인할 범위여서 제가 구현한 범위로 적지 않았습니다.

링크 github.com/mingming-01/sentra | sentra.koreacentral.cloudapp.azure.com

03 Packtory_DE | 제조/물류 업무를 다루는 생산관리(MES형) 시스템

구분/기간	ITWill 심화 과정 팀 프로젝트, 2025.03 ~ 2025.05
사용 기술	Java 21, Spring Boot, JPA/Hibernate, MyBatis, QueryDSL, MySQL, Thymeleaf
개요	인사 정보부터 제품, 원자재, 창고, 입출고, 품질 검사, 근무 관리까지 여러 제조 업무를 연결한 시스템입니다. 이미 만들어진 공통 구조를 이해한 뒤 기능을 추가하고 수정하는 방식으로 개발했고, 팀 서버를 두고 실제 서버 환경에서 시연했습니다.

담당 업무

- 직원 등록·조회·수정·삭제 기능, 대표 부서/하위 부서를 코드로 연결하는 부서 관리, 직급 관리
- 화면의 한글 이름 표시와 데이터베이스의 코드값 저장을 위한 변환 처리
- 기존 Item 중심 구조의 Product 중심 정리와 관련 코드·화면 수정
- 원자재, 창고, 위치, 입출고 기능과 품질 기준, 검사 이력, 근무 스케줄 기능 작업

문제 해결

- **남이 만든 규칙에 맞춰 개발하기** 공통코드와 기존 코드 형식은 처음 접하는 방식이라 적응하는 데 시간이 걸렸습니다. 기능이 동작해도 팀이 이어서 유지할 수 있는 형식이어야 한다는 점을 이 프로젝트에서 알게 됐습니다.
- **데이터베이스 전환 시도** 프로젝트 후반에 MySQL에서 Oracle로 바꾸는 작업을 했지만 수정할 범위가 예상보다 커서 최종 연결까지는 마무리하지 못했습니다. 접속 정보 외에도 SQL, 드라이버, 설정을 함께 고쳐야 한다는 것을 경험했습니다.

결과와 느낀 점

가장 많은 영역을 직접 다룬 프로젝트입니다. 직원과 부서부터 원자재, 입출고, 품질, 근무까지 한 시스템 안에서 이어서 작업하며, 업무 데이터가 시스템 전체에서 어떻게 흐르는지 보게 됐습니다.

링크 github.com/mingming-01/packtory_

보조 프로젝트

04 Mr.Bean | 제조/생산관리 웹 프로젝트

구분/기술	ITWill 팀 프로젝트 | Java 11, Spring MVC, MyBatis, MySQL, JSP. 원자재, 완제품, BOM, 입고와 LOT 번호를 관리하는 웹 프로젝트 github.com/shoqying/MrBean
담당 업무	- 원자재·완제품의 등록, 조회, 수정, 삭제와 완제품 등록 시 BOM 선택 연결 - BOM 관리 (이름 중복 확인, ID 자동 생성, 원자재 연결, 비율 0~100 범위 검증) - 원자재 입고 관리, 날짜 기반 LOT 번호 생성, 입고일 기준 유통기한 계산

문제 해결	**원인을 끝까지 따라가기** 화면에 값이 보이지 않을 때 화면 코드만 고치지 않고 서버 코드, 데이터베이스 처리, SQL을 차례로 따라가며 원인을 찾았습니다.

05 Stock-Mate | QR/바코드 기반 재고 관리 시스템

구분/기술	ITWill 팀 프로젝트 | Java 11, Spring MVC, MyBatis, MySQL, JSP, jQuery. 상품·재고·창고 입출고 관리와 QR/바코드 상품 식별 웹서비스 github.com/in27sung/stock-mate
담당 업무	- 카테고리 등록·조회·수정·삭제와 관련 SQL 작성, 화면 연결, 오류 수정 - 재고 목록 기능 수정, 사용자 정보 수정 오류와 공통 예외 처리 일부 수정
문제 해결	**다른 팀원의 기능과 맞물리는 데이터** 카테고리는 상품이 쓰고, 재고는 상품과 창고에 연결되며 주문과 입출고도 같은 데이터를 참조합니다. 제 기능을 고칠 때 다른 팀원이 그 데이터를 어떻게 쓰는지 함께 확인해야 했고, Git 브랜치와 병합으로 작업을 합쳤습니다.

ALEPH 과제와 개인 실습

ALEPH 보안 과제	**보안 방어전** Supabase 로그인과 토큰 검증, 소유자 검사, 등록 기기 판정, 백업 한 건 복원 API 등 단계별 구현 (Vercel). 판정과 탐지는 로컬 가상 요청으로 확인했고 실제 운영 환경의 권한 회수는 하지 않음 **내 보안 Agent** 방어전 결과를 확인하는 로컬 실행 앱 (배포하지 않음) | **XDR 웹 주입 탐지** 판정 규칙 작성 | **My Profile** 패스키(WebAuthn) 로그인 학습용 프로토타입
개인 실습	**T10 타이핑 연구** 각 15회, 총 30회 측정과 Python 분석. 표본이 작아 관찰한 범위에서만 해석 (2026.10) | **PDS Diary** Flask, SQLite, 자동 테스트 15개 | **BR-A 포트폴리오** GitHub Pages